pfi

Python for Fixed Income

COURSE 5 · ADVANCED MACHINE LEARNING

Impurity and Permutation Importance

WEEK 1 · DAY 4

Created by **Jeff Lenamon**

This course is educational content created in a personal capacity. Nothing here is investment advice or a recommendation to buy or sell any security. All portfolio data is synthetic.

Which columns does the forest lean on?

Two standard answers

Impurity importance: feature_importances_, free with the fit, sums to 1.

Permutation importance: break one column on unseen rows and measure the drop.

Today's test

Add a column of random numbers. Its worth is known: nothing. Which measure notices?

Before you read a feature importance chart, ask which measure it is and which rows it came from.

How impurity importance works

The problem	Which columns does a fitted forest lean on? The quickest answer is already inside it
Credit	Each split lowers the Gini impurity at its node; credit the decrease to the split's column
Add up	Weight by the share of rows reaching the node, sum over a tree, average over trees, scale to 1
You control	Nothing: it is worked out during the fit, from the training rows only
What comes out	feature_importances_: one share per column, a description of how the forest split its rows

Two importances, two different questions

Impurity importance

Each split lowers the Gini impurity of the training rows it splits. Credit the decrease to the split's column, weighted by the rows at the node; add over the tree, average over trees, scale to sum to 1.

From the training rows, during the fit

Permutation importance

Shuffle one column so its values land on the wrong accounts, score the model again, record the drop in validation ROC AUC. Repeat five times and average.

On rows the model never saw

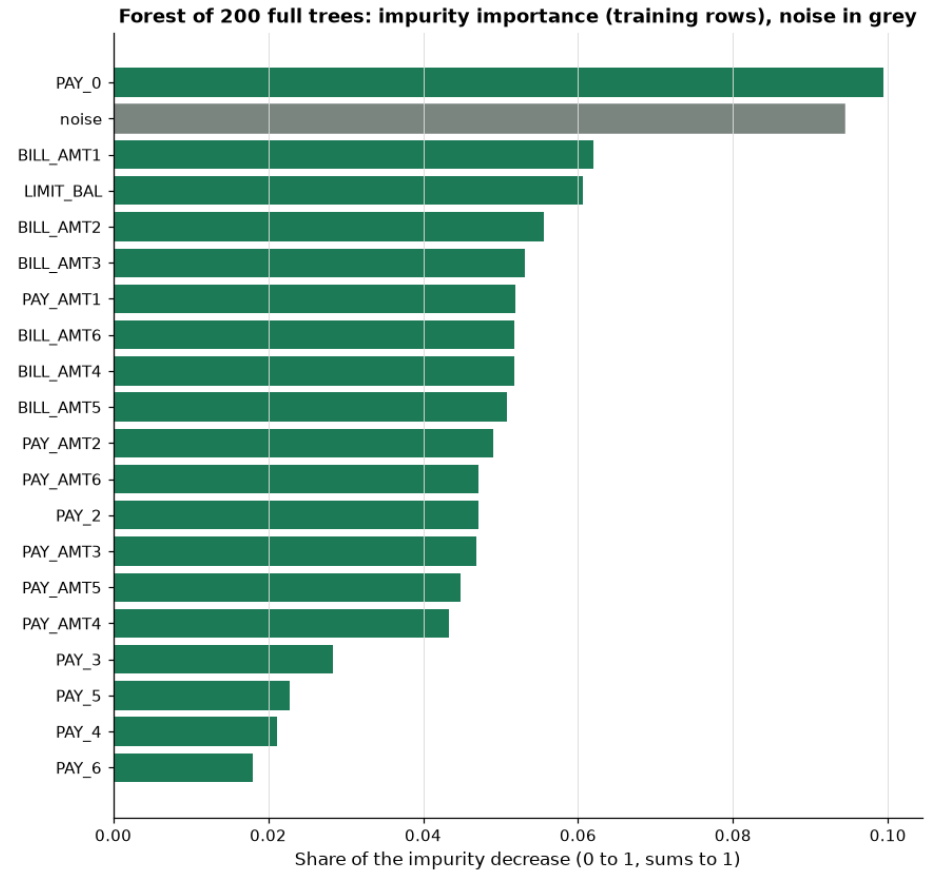
```
table = ensemblekit.importance_table(forest, X_valid, y_valid)
```

One is a byproduct of fitting; the other is an experiment on unseen rows. 18 tests pass today.

How the noise-column test works

The problem	How do you check an importance measure? Give it a column that cannot matter
Add	Random numbers tied to no account, as a column named noise, on training and validation rows
Refit and read	A measure of what the forest learned about defaults should give it close to nothing
You control	The column's distribution and its seed
What comes out	A rank and a score for noise beside every real column

A column of random numbers ranks near the top



2nd of 20

noise, by impurity importance

0.0943

its impurity importance

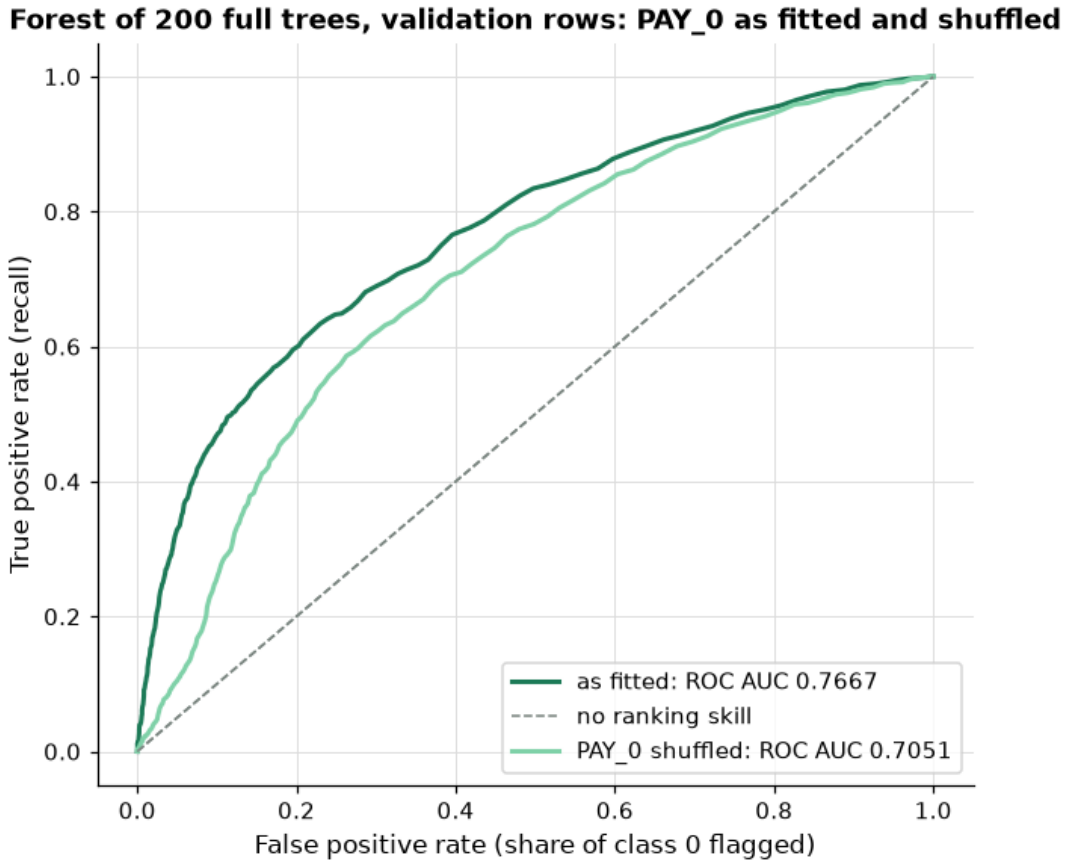
noise: one normal random number per row, training and validation. Correlation with the outcome -0.0086. The forest of 200 full trees: validation ROC AUC 0.7667, average precision 0.5224.

Impurity importance measures how much a column was used to split the training rows.

How permutation importance works

The problem	Impurity importance describes how the forest was built; how much does it need a column?
Shuffle	Score unseen rows, shuffle one column so its values go to the wrong accounts, score again
Importance	The fall in the score; repeat the shuffle and average to steady it
You control	The rows, the score, and the number of shuffles
What comes out	A mean drop and its spread for each column

One shuffle of PAY_0: the area between two curves



0.7667

as fitted

0.7051

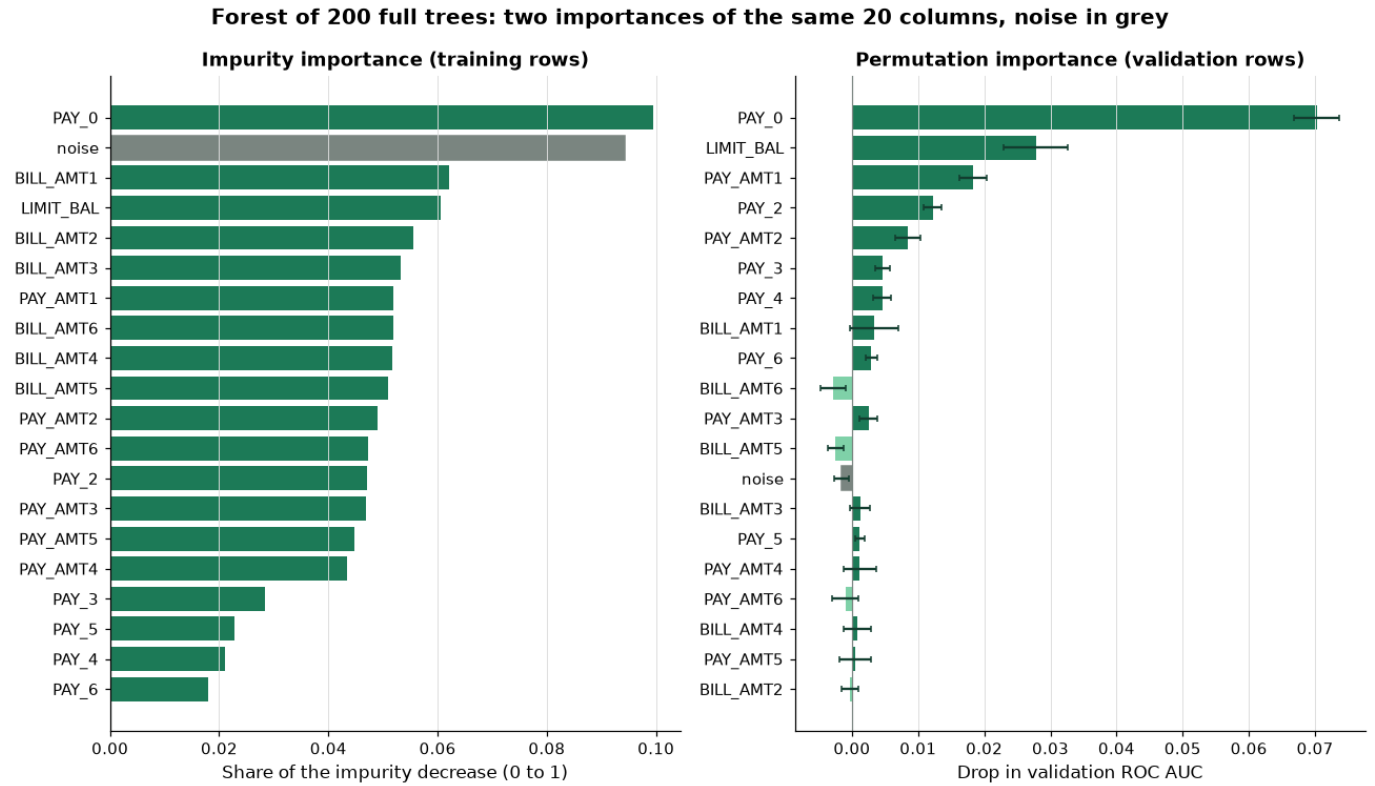
PAY_0 shuffled

The drop: 0.0616

Same trees, the column handed to the wrong accounts. The drop in ROC AUC is the area between the two curves.

One shuffle is one draw: importance_table shuffles each column five times.

Permutation importance on validation rows



-0.0015

noise: permutation importance

0.0011

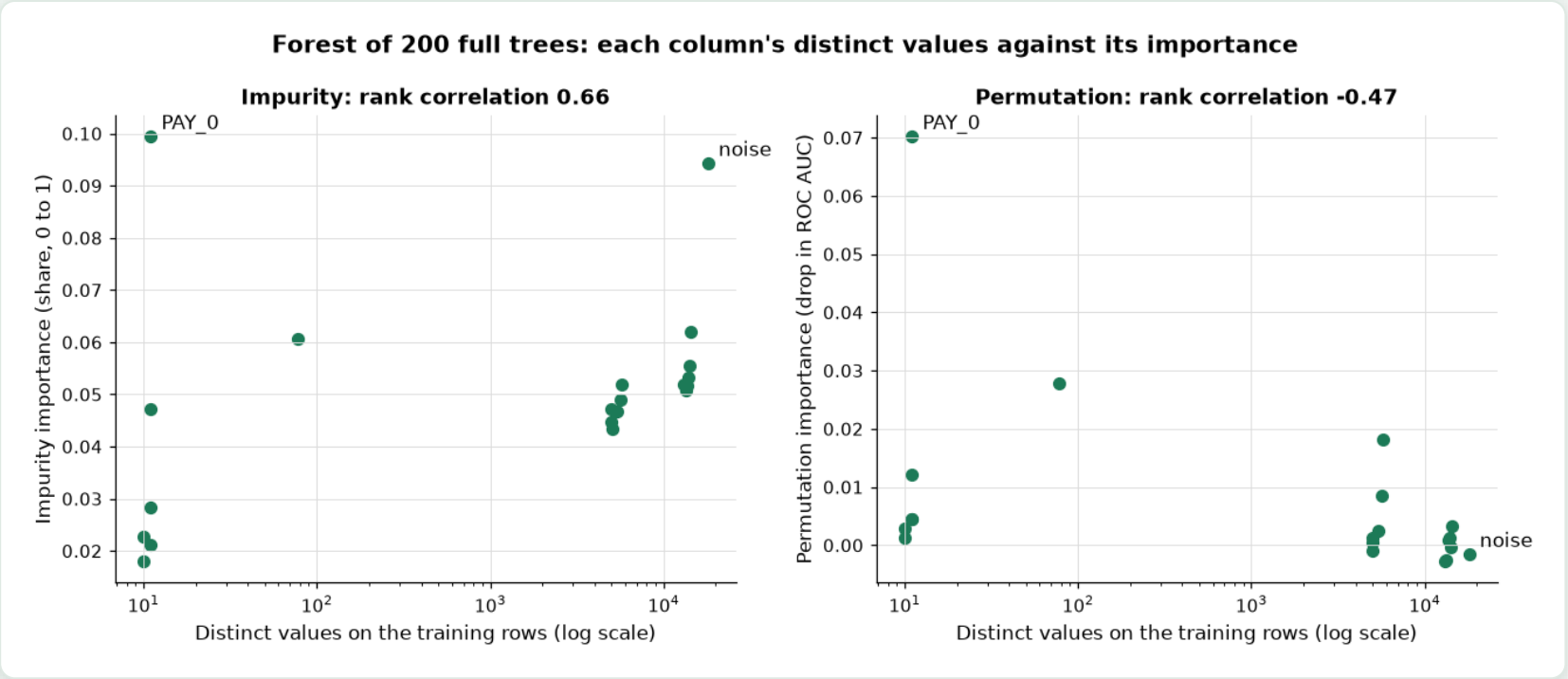
standard deviation, five shuffles

18th of 20

its permutation rank

PAY_0 0.0702, LIMIT_BAL 0.0278. noise: within two standard deviations of zero.

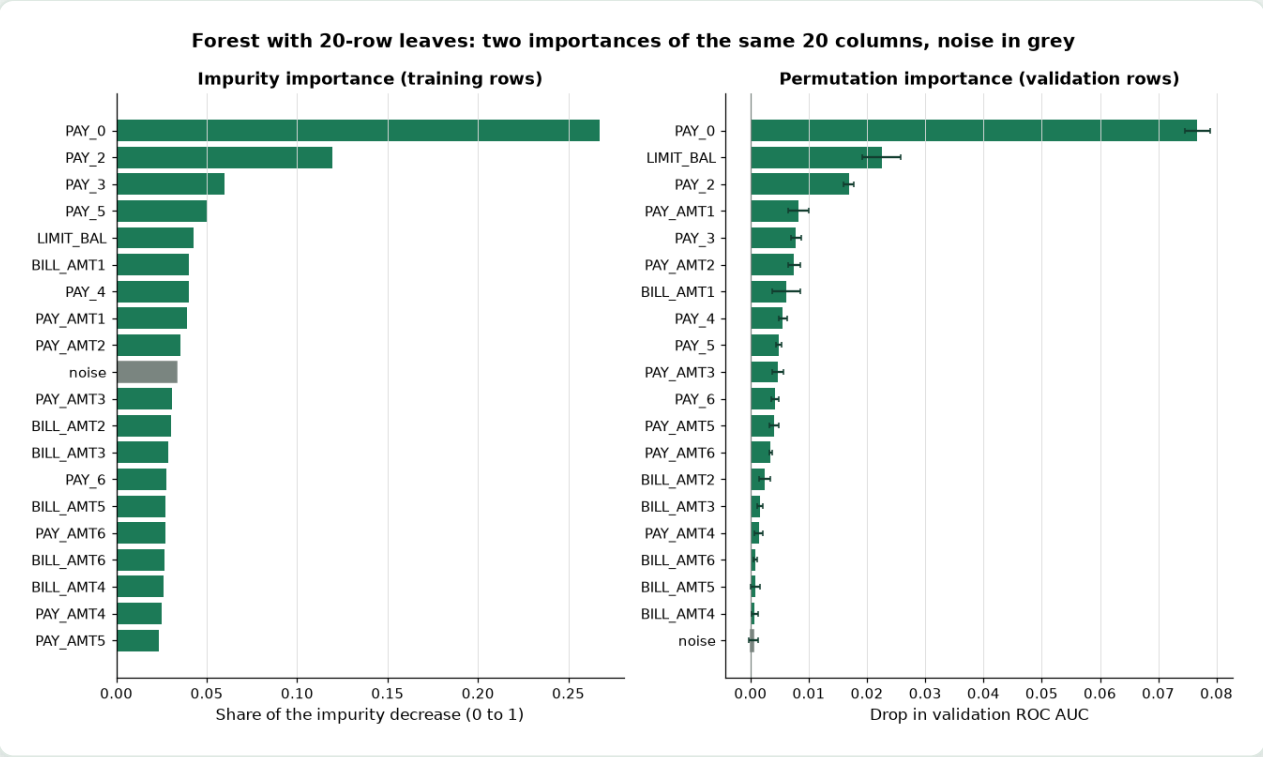
Why impurity importance rewards the noise column



Column	Distinct
noise	18,000
BILL_AMT1	14,404
PAY_AMT1	5,717
PAY_0	11

Deep in a full tree a node holds a handful of rows; a column with thousands of values offers thousands of thresholds, and one splits a few defaults off by chance.

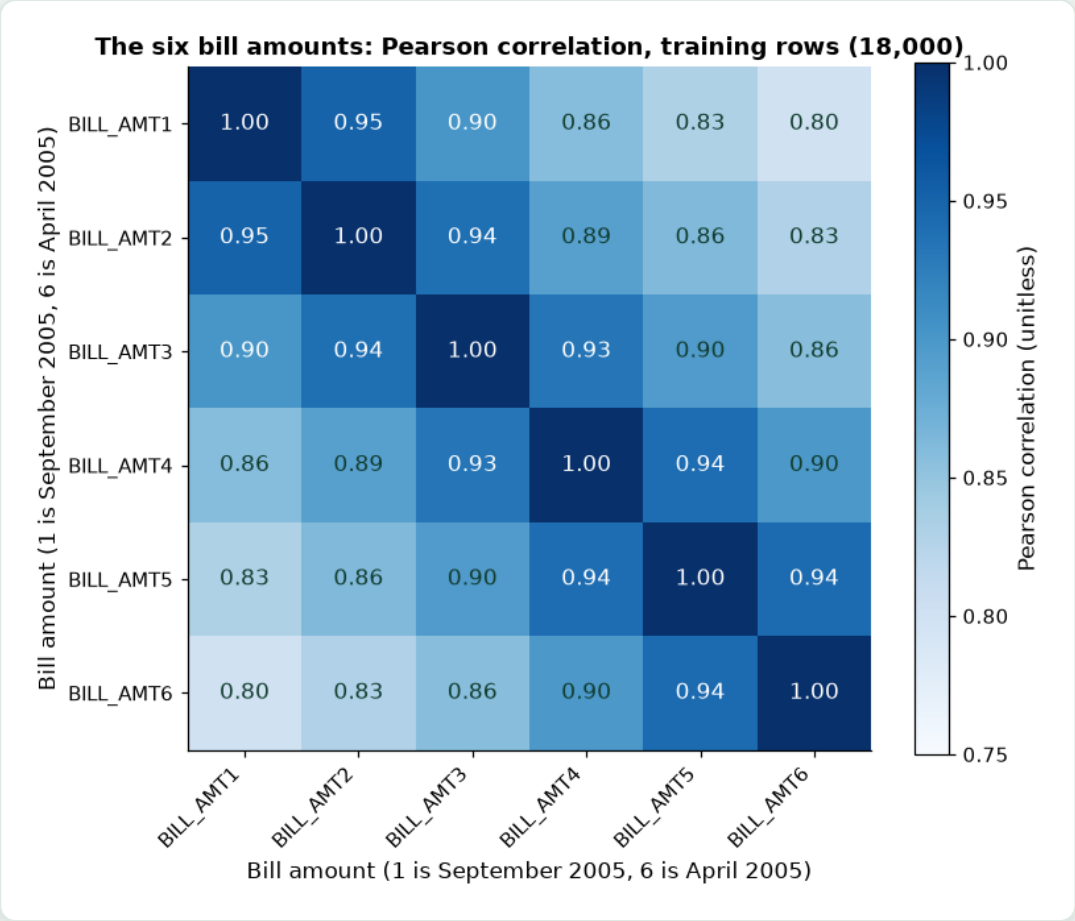
Leaves of 20 rows shrink the bias, not to zero



	Full trees	20-row leaves
Validation ROC AUC	0.7667	0.7861
noise impurity	0.0943	0.0333
its rank	2nd	10th
noise permutation	-0.0015	0.0005

Impurity importance is never zero for a column a tree can split on: it cannot say a column is worthless.

Correlated columns share importance

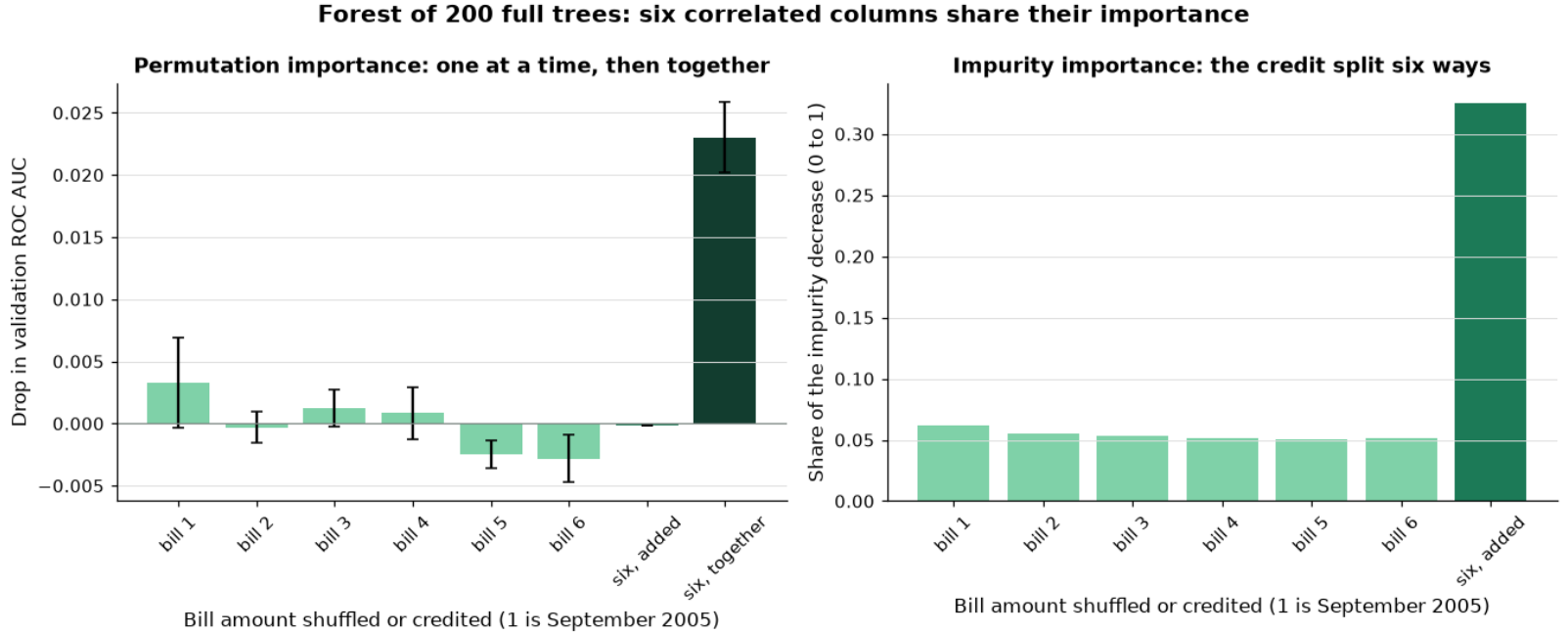


BILL_AMT1 (September 2005) to BILL_AMT6 (April 2005): every pair correlates between 0.80 and 0.95 on the training rows. Neighbouring months most.

```
broken[bills] = X_valid[bills].to_numpy()[order]
```

Break one, and the forest still has five near-copies. Break all six with one order of rows, and it has none.

One at a time, or all six together



-0.0002

one at a time, added

0.0230

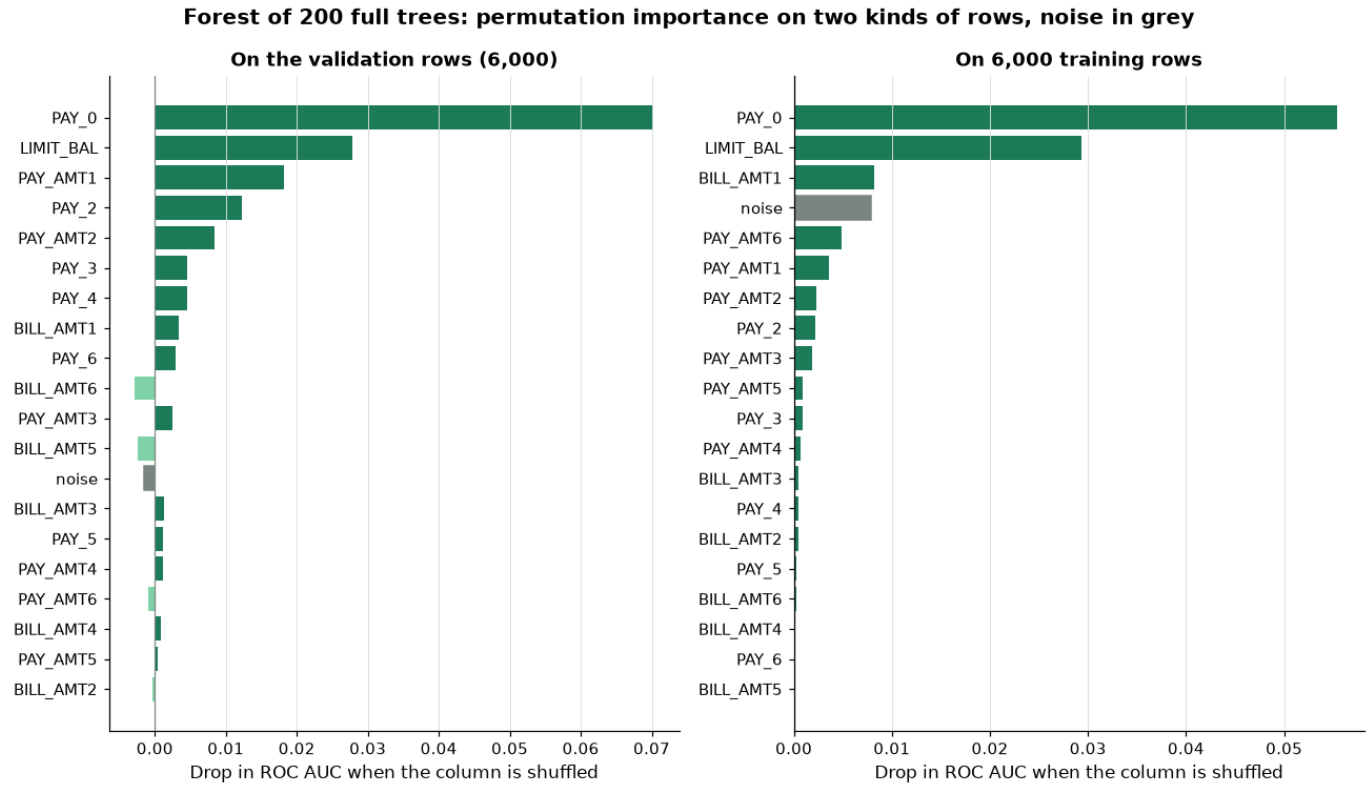
together, sd 0.0028

0.3252

impurity, added

Permutation importance of one column is what the model loses when that column alone is broken.

Permutation importance on training rows



0.0079

noise on training rows

-0.0015

noise on validation rows

What it measures there

ROC AUC 1.0000 on those rows. Rank 4th of 20: how much the model used a column to fit them, noise included.

Every time you are shown permutation importance, ask: on which rows?

What to call an importance

Allowed names

impurity importance

permutation importance: the drop in

validation ROC AUC when a column is shuffled

the model leans on; association in this file

Never called

driver, key driver

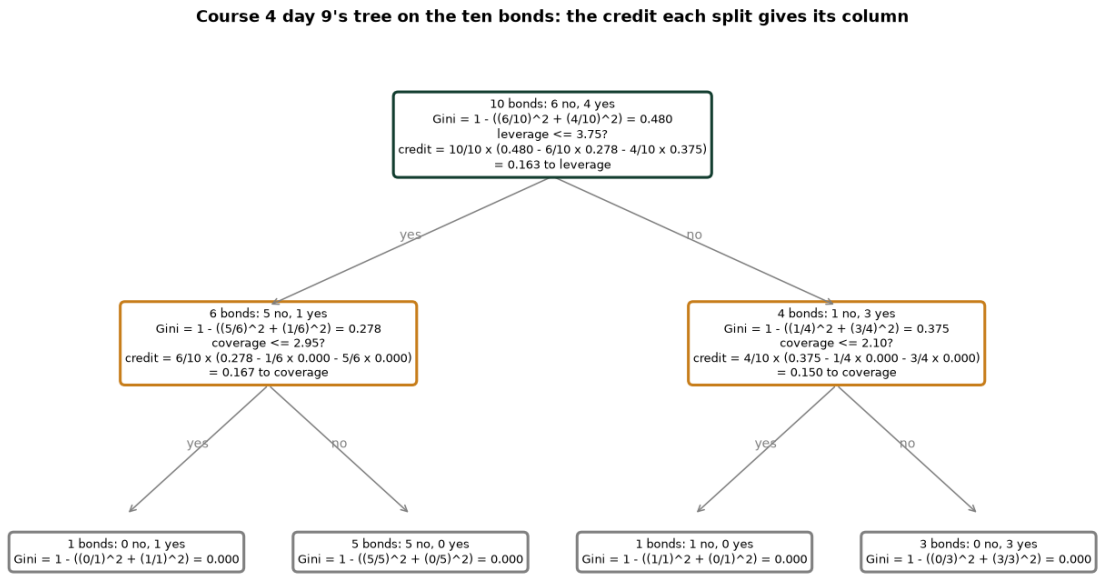
cause or determinant

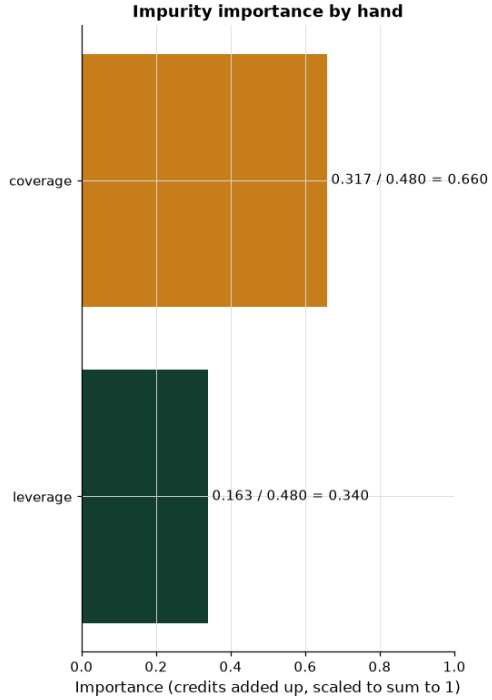
risk factor

PAY_0 first on permutation importance: this forest ranks these accounts much worse without it. Nothing about what would happen if an account's payment status changed.

Today: random numbers ranked 2nd by one measure; six related columns near zero by the other.

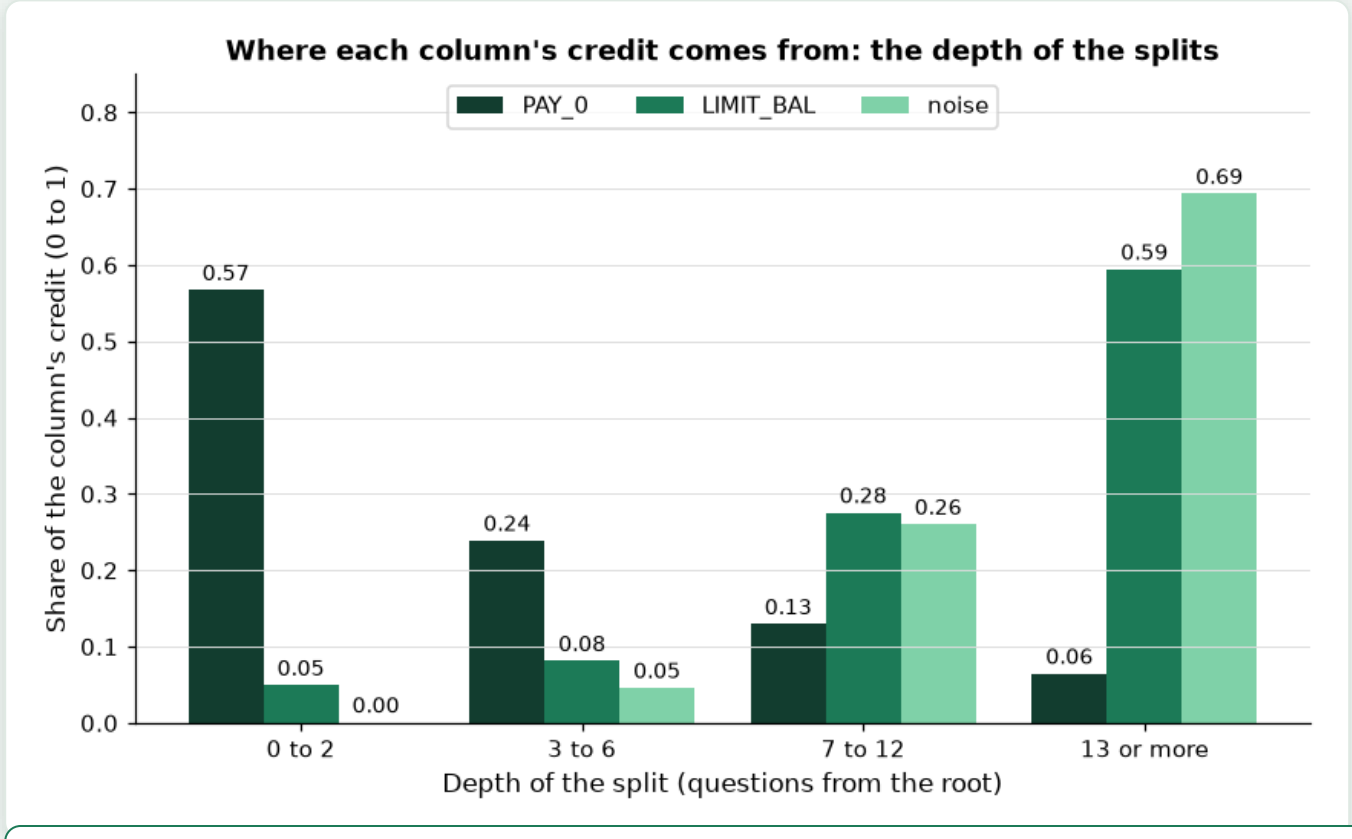
Impurity importance is a sum of Gini decreases





Credits added up: leverage 0.1633, coverage 0.3167. Scaled to sum to 1: 0.3403 and 0.6597, equal to feature_importances_. The same sum over 501,277 splits gives the forest's.

Where each column earns its credit



Depth of split	PAY_0	LIMIT_BAL	noise
0 to 2	0.567	0.049	0.001
3 to 6	0.239	0.081	0.046
7 to 12	0.129	0.275	0.260
13 or more	0.065	0.594	0.693

PAY_0 earns its credit at the top, where boxes hold thousands of accounts. noise earns its credit at the bottom, in boxes of a handful.

A full tree makes huge numbers of tiny deep splits, and a column of random numbers earns a little from each. Permutation importance sums nothing over training rows.

Excel side by side

Excel side by side (tested in Excel)

`=RANK.EQ(B2,$B$2:$B$21)`	noise ranks 2
`=RANK.EQ(C2,$C$2:$C$21)`	noise ranks 18
`=RANK.AVG(B2,$B$2:$B$6001,1)`	each output's rank, ties averaged
`=COUNTIF(A2:A6001,1), =COUNTIF(A2:A6001,0)`	1,327 and 4,673
`=(SUMIF(A2:A6001,1,D2:D6001)-H1*(H1+1)/2)/(H1*H2)`	ROC AUC 0.7667; shuffled 0.7051
`=CORREL(A2:A18001,B2:B18001)`	0.9511, September with August

No forest, no impurity importance, and no seeded shuffle in Excel. The bars: a clustered bar chart; the heatmap: a colour scale.

Save the day: HEAD~1

1. Code

mlkit.py, ensemblekit and its tests

2. Log

results.csv: the full-tree forest

3. Log again

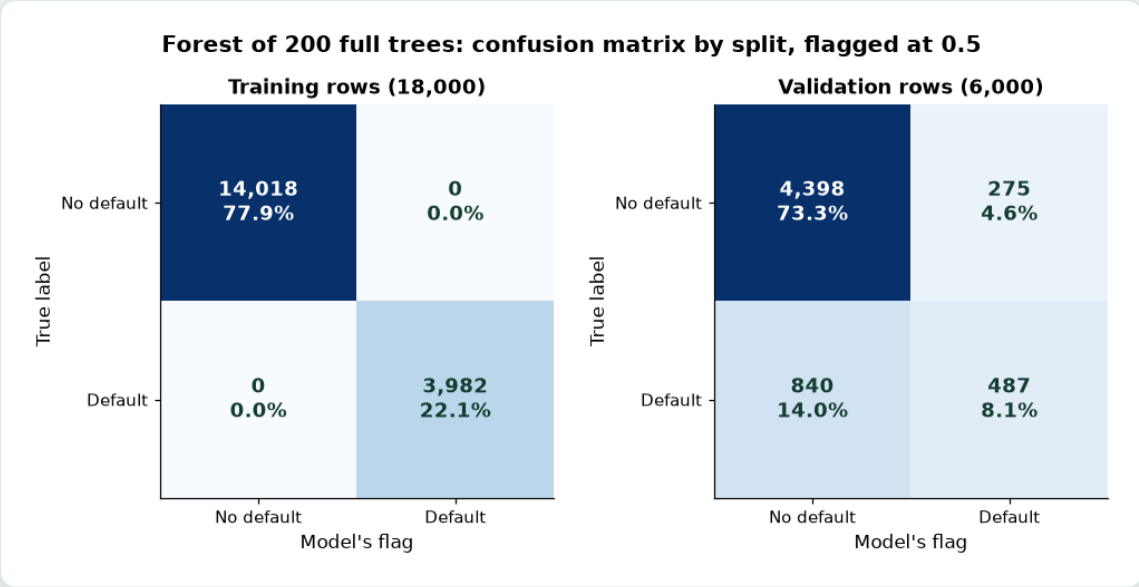
add the 20-row-leaf forest

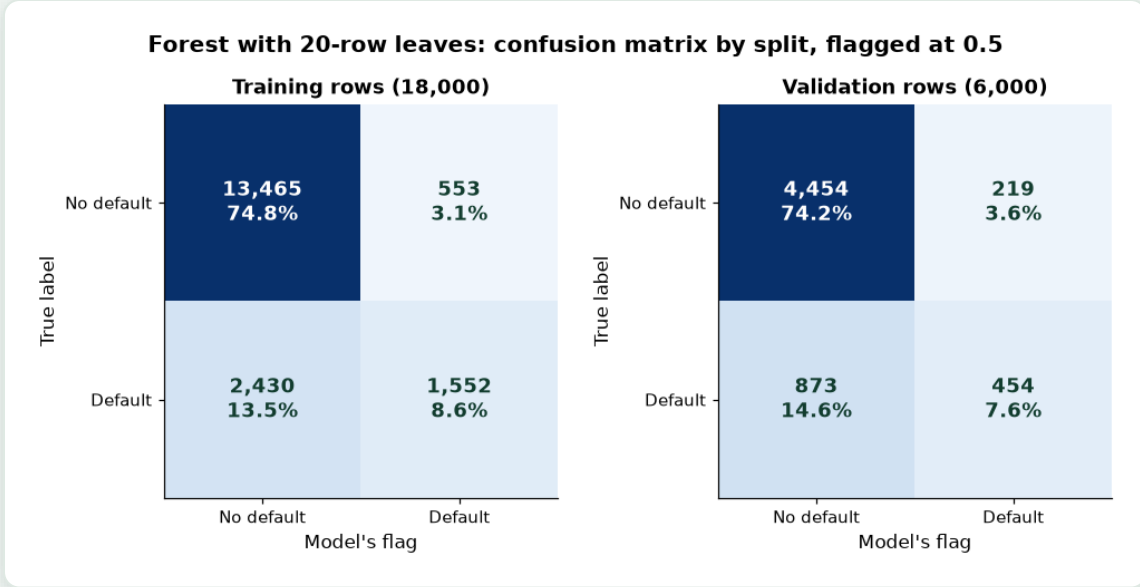
```
!git -C "{work}" show HEAD~1:results.csv
```

```
model,rows,roc_auc,average_precision,threshold,accuracy,precision,recall,f1
forest_200_full_trees_noise,validation,0.7667,0.5224,0.5000,0.8142,0.6391,0.3670,0.4663
```

HEAD~1:file prints the file one commit back and changes nothing. All-zero baseline 0.7788, never to choose.

The two logged forests, by split





Validation rows, flags at 0.5	ROC AUC	Precision	Recall	F1
Full trees	0.7667	0.6391	0.3670	0.4663
20-row leaves	0.7861	0.6746	0.3421	0.4540

Training panels show rows the trees were fitted on: the same lesson as 4.6, in counts.

The AI check

The docstring is the prompt. The code that comes back runs, and its number looks fine.

1

Docstring

Which rows fit, which rows choose, which rows report: what you asked for.

→

2

Code back

Written for the course in the style of an AI answer. Read it.

→

3

Test first

Assert what the docstring promises. Run it: it fails.

→

4

Fix

Find the rule it broke, fix it, run the test again.

Today: it shuffles the training rows: the noise column looks important on new accounts.

Code that runs is not the same as code that is right.

What you can do now

- Read impurity and permutation importance side by side
- Test a measure with a column of random numbers
- Full trees inflate impurity importance for many-valued columns
- Shuffle correlated columns together
- Ask of permutation importance: on which rows?

Week 1, Day 5 Case study: forests and the spread cross-section